

Spring Boot Online Judge System - Complete Documentation

Table of Contents

1. [System Overview](#)
 2. [Architecture](#)
 3. [Configuration Files](#)
 4. [Entity Layer](#)
 5. [Repository Layer](#)
 6. [Service Layer](#)
 7. [Controller Layer](#)
 8. [Request/Response Flow](#)
 9. [Validation System](#)
 10. [File Dependencies](#)
-

System Overview

What This System Does

This is an **Online Judge System** for evaluating Spring Boot code submissions. It allows students to submit code for specific tasks in two project types:

- **Todo Application** (5 tasks)
- **Blog Application** (6 tasks)

The system validates code against predefined rules, checks for proper annotations, methods, fields, and can even compile the code to verify syntax.

Key Features

- Code submission and evaluation
 - Real-time validation against predefined rules
 - Support for multiple programming tasks
 - Compilation checking (when javac is available)
 - Submission history tracking
 - RESTful API endpoints
-

Architecture



Configuration Files

1. application.properties

Location: src/main/resources/application.properties

Purpose: Configures the Spring Boot application settings

Key Configurations:

properties

Application Identity

`spring.application.name=springboot-judge`

`server.port=8080`

Database (H2 In-Memory)

`spring.datasource.url=jdbc:h2:mem:judgetb`

`spring.datasource.username=sa`

`spring.datasource.password=`

JPA Settings

`spring.jpa.hibernate.ddl-auto=update` # Auto-create tables

`spring.jpa.show-sql=true` # Log SQL queries

File Upload Limits

`spring.servlet.multipart.max-file-size=10MB`

Judge-Specific Settings

`judge.temp.directory=/tmp/springboot-judge`

`judge.timeout.seconds=30`

2. `pom.xml` Dependencies

Key Dependencies:

- `spring-boot-starter-web` → REST API support
- `spring-boot-starter-data-jpa` → Database operations
- `h2` → In-memory database
- `lombok` → Reduces boilerplate code
- `spring-boot-starter-validation` → Input validation

Entity Layer

1. `Submission.java`

Purpose: Represents a code submission in the database

Fields:

java

- id (**Long**) → **Primary** key
- projectType (**String**) → "todo" or "blog"
- taskId (**Integer**) → Which task (1-5 for todo, 1-6 for blog)
- code (**String**) → Submitted code (max 10,000 chars)
- status (**String**) → PENDING, RUNNING, PASSED, FAILED
- output (**String**) → Evaluation results (max 5,000 chars)
- testsPassed (**Integer**) → Number of tests passed
- testsTotal (**Integer**) → Total number of tests
- submittedAt (**DateTime**) → Submission timestamp
- evaluatedAt (**DateTime**) → Evaluation completion timestamp

Key Annotation:

```
java
@PrePersist
public void prePersist() {
    submittedAt = LocalDateTime.now();
    status = "PENDING";
}
```

This method automatically runs before saving, setting initial values.

2. Task.java

Purpose: Stores task definitions (currently not actively used, but designed for extensibility)

Fields:

```
java
- id, projectType, taskNumber
- title, description, instructions
- starterCode → Template code for students
- testCode → JUnit test code
- validationRules → JSON format validation rules
```

Repository Layer

1. SubmissionRepository.java

Purpose: Database access for submissions

Type: Interface extending JpaRepository

Custom Method:

```
java
```

```
List<Submission> findByProjectTypeAndTaskIdOrderBySubmittedAtDesc(  
    String projectType,  
    Integer taskId  
);
```

What it does: Retrieves all submissions for a specific project type and task, ordered by newest first.

How Spring generates it: Spring Data JPA automatically implements this method based on the method name pattern.

2. `TaskRepository.java`

Custom Methods:

```
java
```

```
Optional<Task> findByProjectTypeAndTaskNumber(String projectType, Integer taskNumber);  
List<Task> findByProjectTypeOrderByTaskNumber(String projectType);
```

Service Layer

1. `ValidationRuleProvider.java`

Purpose: Provides validation rules for each task

Main Method:

```
java
```

```
public List<ValidationRule> getRulesForTask(String projectType, Integer taskId)
```

How it works:

1. Takes project type ("todo" or "blog") and task ID
2. Uses switch statement to determine which rules to apply
3. Returns a list of `ValidationRule` objects

Example Rules for Todo Entity (Task 1):

```
java
```

- Check for @Entity annotation
- Check for @Id annotation
- Check for @GeneratedValue annotation
- Verify id field of type Long exists
- Verify title, description, completed fields exist

Rule Types:

- **ANNOTATION** → Checks for specific annotations
- **FIELD** → Checks for field declarations using regex
- **METHOD** → Checks for method signatures
- **CONTAINS** → Simple string containment check
- **REGEX** → Pattern matching

2. **CodeEvaluationService.java**

Purpose: The core evaluation engine

Main Method:

```
java  
  
public List<TestResult> evaluateCode(String code, String projectType, Integer taskId)
```

Evaluation Flow:

1. Create temp directory for code files
↓
2. Get validation rules from ValidationRuleProvider
↓
3. For each rule:
 - Run validateRule()
 - Create TestResult
 - Add to results list↓
4. If code looks complete, run compilation check
↓
5. Return all test results

Key Methods:

validateRule(String code, ValidationRule rule)

```
java
```

```

switch (rule.getType()) {
    case "CONTAINS":
        return code.contains(rule.getPattern());

    case "REGEX":
        Pattern pattern = Pattern.compile(rule.getPattern());
        return pattern.matcher(code).find();

    case "ANNOTATION":
        return checkAnnotation(code, rule.getPattern());

    case "METHOD":
        return checkMethod(code, rule.getPattern());

    case "FIELD":
        return checkField(code, rule.getPattern());
}

```

checkAnnotation(String code, String annotation)

- Simply checks if code contains "@" + annotation name

checkMethod(String code, String methodSignature)

- Removes extra whitespace
- Checks if method signature exists in code

checkField(String code, String fieldPattern)

- Uses regex to find field declarations
- Pattern: `((private|public|protected)\s+fieldPattern)`
- Example: Finds `private Long id`

checkCompilation(String code)

This is the most complex method:

```

java

```

1. Extract **class** name using regex: `"class\s+(\w+)"`



2. Create temporary .java file in temp directory



3. Try to compile using javac command

```
ProcessBuilder pb = new ProcessBuilder("javac", tempFile);
```



4. Capture compilation output



5. If `exitCode == 0` → Success

If `exitCode != 0` → Compilation errors



6. If javac not available, fall back to basic syntax check



7. Clean up temporary files

checkBasicSyntax(String code)

- Counts opening/closing braces `{ }`
- Counts opening/closing parentheses `( )`
- Returns true if they match

3. SubmissionService.java

Purpose: Orchestrates the submission process

Dependencies:

java

`@RequiredArgsConstructor // Lombok generates constructor`

```
private final SubmissionRepository submissionRepository;
```

```
private final CodeEvaluationService evaluationService;
```

Main Method: `submitCode(SubmitCodeRequest request)`

Step-by-Step Flow:

java

1. CREATE SUBMISSION ENTITY

```
Submission submission = new Submission();
submission.setProjectType(request.getProjectType());
submission.setTaskId(request.getTaskId());
submission.setCode(request.getCode());
submission.setStatus("RUNNING");
submission = submissionRepository.save(submission);
```

2. EVALUATE CODE

```
List<TestResult> testResults = evaluationService.evaluateCode(
    request.getCode(),
    request.getProjectType(),
    request.getTaskId()
);
```

3. CALCULATE RESULTS

```
long passedCount = testResults.stream()
    .filter(TestResult::getPassed)
    .count();
int totalTests = testResults.size();
boolean allPassed = (passedCount == totalTests);
```

4. BUILD OUTPUT STRING

```
StringBuilder output = new StringBuilder();
if (allPassed) {
    output.append("✅ All tests passed! Excellent work!\n\n");
} else {
    output.append("❌ Some tests failed:\n\n");
}

for (TestResult result : testResults) {
    output.append(result.getMessage()).append("\n");
    if (!result.getPassed() && result.getExpected() != null) {
        output.append("    Expected pattern: ")
            .append(result.getExpected()).append("\n");
    }
}
```

5. UPDATE SUBMISSION

```
submission.setStatus(allPassed ? "PASSED" : "FAILED");
submission.setTestsPassed((int) passedCount);
submission.setTestsTotal(totalTests);
submission.setOutput(output.toString());
submission.setEvaluatedAt(LocalDateTime.now());
submission = submissionRepository.save(submission);
```

6. BUILD AND RETURN RESPONSE

```
SubmissionResponse response = new SubmissionResponse();
response.setSubmissionId(submission.getId());
response.setStatus(submission.getStatus());
response.setOutput(submission.getOutput());
response.setTestsPassed(submission.getTestsPassed());
response.setTestsTotal(submission.getTestsTotal());
response.setAllPassed(allPassed);
return response;
```

Other Methods:

getSubmission(Long id)

```
java

return submissionRepository.findById(id)
    .orElseThrow(() -> new RuntimeException("Submission not found"));
```

Retrieves a single submission by ID, throws exception if not found.

getSubmissions(String projectType, Integer taskId)

```
java

return submissionRepository.findByProjectTypeAndTaskIdOrderBySubmittedAtDesc(
    projectType, taskId
);
```

Retrieves all submissions for a specific task.

Controller Layer

JudgeController.java

Purpose: REST API endpoints for the system

Base Path: /api/judge

Annotations:

```
java
```

```
@RestController // Combines @Controller + @ResponseBody
@RequestMapping("/api/judge")
@RequiredArgsConstructor // Constructor injection
@CrossOrigin(origins = "**") // Allow CORS from any origin
```

Endpoints:

1. Submit Code

java

POST /api/judge/submit

```
@PostMapping("/submit")
public ResponseEntity<SubmissionResponse> submitCode(
    @RequestBody SubmitCodeRequest request
)
```

Request Body:

```
{
  "projectType": "todo",
  "taskId": 1,
  "code": "public class TodoEntity { ... }"
}
```

Response:

```
{
  "submissionId": 123,
  "status": "PASSED",
  "output": "✅ All tests passed!...",
  "testsPassed": 7,
  "testsTotal": 7,
  "allPassed": true
}
```

2. Get Single Submission

java

```
GET /api/judge/submission/{id}
```

```
@GetMapping("/submission/{id}")
```

```
public ResponseEntity<Submission> getSubmission(@PathVariable Long id)
```

Example: GET /api/judge/submission/123

Response: Full Submission entity

3. Get All Submissions for Task

```
java
```

```
GET /api/judge/submissions?projectType=todo&taskId=1
```

```
@GetMapping("/submissions")
```

```
public ResponseEntity<List<Submission>> getSubmissions(
```

```
    @RequestParam String projectType,
```

```
    @RequestParam Integer taskId
```

```
)
```

Response: Array of Submission entities

4. Health Check

```
java
```

```
GET /api/judge/health
```

Response: "Spring Boot Online Judge is running!"

Request/Response Flow

Complete Flow for Code Submission

1. CLIENT SENDS REQUEST

```
POST /api/judge/submit
```

```
{  
    "projectType": "todo",  
    "taskId": 1,  
    "code": "..."  
}
```

2. JUDGECONTROLLER RECEIVES REQUEST

```
submitCode(@RequestBody SubmitCodeRequest request)
```

↓

```
Calls: submissionService.submitCode(request)
```

3. SUBMISSIONSERVICE PROCESSES

- a. Creates new Submission entity
- b. Saves to database (status: RUNNING)
- c. Calls: `evaluationService.evaluateCode(code, projectType, taskId)`

4. CODEEVALUATIONSERVICE EVALUATES

- a. Calls: `ruleProvider.getRulesForTask(projectType, taskId)`
- b. For each rule:
 - Calls: `validateRule(code, rule)`
 - Creates `TestResult`
- c. If applicable, calls: `checkCompilation(code)`
- d. Returns: `List<TestResult>`

5. VALIDATIONRULEPROVIDER RETURNS RULES

`getRulesForTask()` returns list of `ValidationRule` objects

6. SUBMISSIONSERVICE CONTINUES

- a. Calculates pass/fail counts
- b. Builds output message
- c. Updates submission entity
- d. Saves to database
- e. Creates `SubmissionResponse`
- f. Returns response

7. JUDGECONTROLLER RETURNS RESPONSE

```
ResponseEntity.ok(response)
```

8. CLIENT RECEIVES RESPONSE

```
{
  "submissionId": 123,
  "status": "PASSED",
  "output": "✅ All tests passed!...",
  ...
}
```

Validation System

How Validation Rules Work

Example: Todo Entity (Task 1)

When a student submits code for creating a Todo entity, the system checks:

Rule 1: `@Entity` annotation

```
java

ValidationRule rule = new ValidationRule(
    "ANNOTATION",      // type
    "@Entity",         // pattern
    "Has @Entity annotation", // description
    "Missing @Entity annotation" // errorMessage
);

// Validation:
checkAnnotation(code, "@Entity")
→ Returns: code.contains("@Entity")
```

Rule 2: `id` field

```
java

ValidationRule rule = new ValidationRule(
    "FIELD",
    "Long\\s+id",
    "Has id field of type Long",
    "Missing id field"
);

// Validation:
checkField(code, "Long\\s+id")
→ Pattern: "(private|public|protected)\\s+Long\\s+id"
→ Returns: pattern.matcher(code).find()
```

Rule 3: `findByCompleted` method (Task 2 - Repository)

```
java

ValidationRule rule = new ValidationRule(
    "METHOD",
    "findByCompleted",
    "Has findByCompleted method",
    "Missing findByCompleted method"
);

// Validation:
checkMethod(code, "findByCompleted")
→ Cleans whitespace and checks if method name exists
```

Test Result Generation

After validation, each rule produces a `TestResult`:

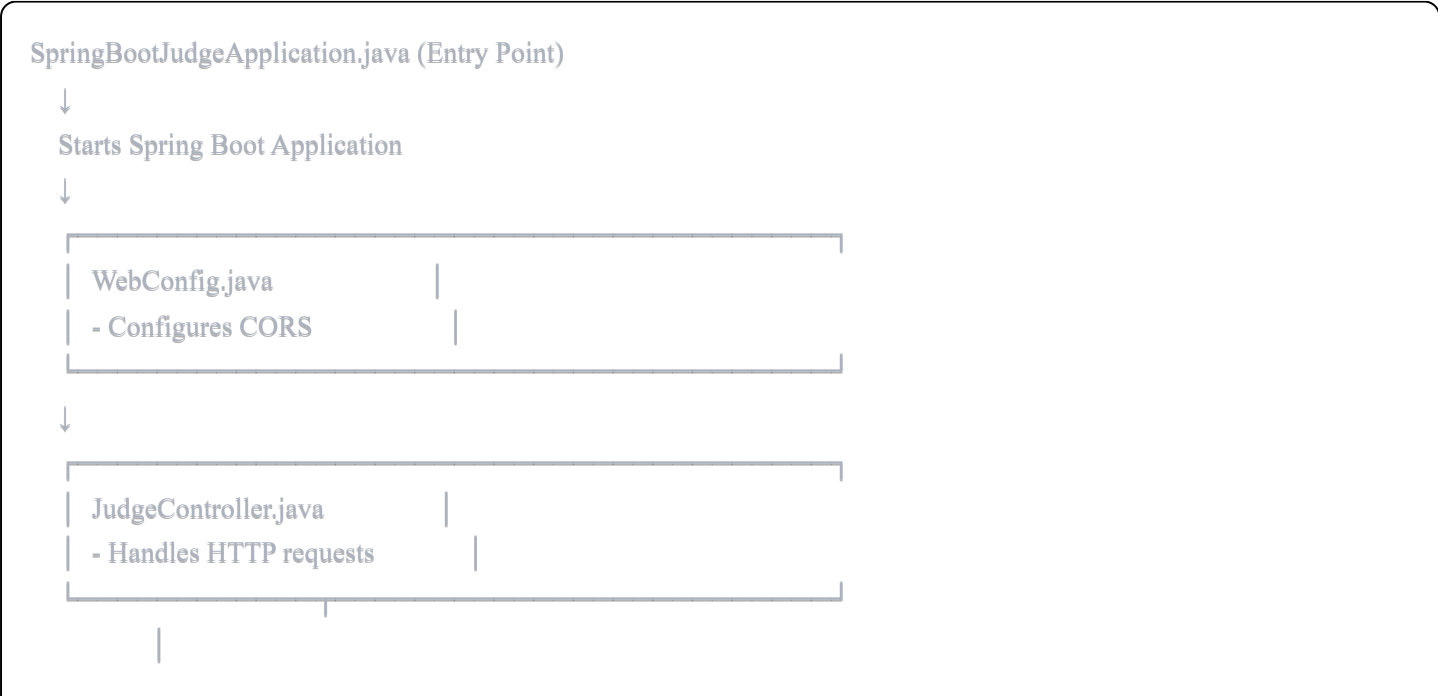
```
java
TestResult {
  testName: "Has @Entity annotation"
  passed: true
  message: "✓ Has @Entity annotation"
  expected: "@Entity"
  actual: null
}
```

If failed:

```
java
TestResult {
  testName: "Has @Entity annotation"
  passed: false
  message: "X Missing @Entity annotation"
  expected: "@Entity"
  actual: null
}
```

File Dependencies

Dependency Graph





DTOs (Data Transfer Objects)

Purpose: Transfer data between layers without exposing entities

SubmitCodeRequest.java

```
java
{
    projectType: String
    taskId: Integer
    code: String
}
```

Used to receive code submissions from clients.

SubmissionResponse.java

```
java
```



```
{  
  submissionId: Long  
  status: String  
  output: String  
  testsPassed: Integer  
  testsTotal: Integer  
  allPassed: Boolean  
}
```

Returned after code evaluation.

TestResult.java

```
java  
  
{  
  testName: String  
  passed: Boolean  
  message: String  
  expected: String  
  actual: String  
}
```

Represents individual test results.

Key Design Patterns

1. Dependency Injection

```
java  
  
@RequiredArgsConstructor // Lombok annotation  
private final SubmissionService submissionService;
```

Spring automatically injects dependencies through constructor.

2. Repository Pattern

```
java  
  
interface SubmissionRepository extends JpaRepository<Submission, Long>
```

Abstracts database operations.

3. Service Layer Pattern

Business logic separated from controllers and repositories.

4. DTO Pattern

Separate objects for data transfer vs. database entities.

5. Strategy Pattern (in validation)

Different validation strategies based on rule type (ANNOTATION, FIELD, METHOD, etc.).

Example Execution Trace

Let's trace a complete submission:

Student submits Todo Entity code:

```
java
// Student's code
@Entity
public class TodoEntity {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String title;
    private String description;
    private boolean completed;
}
```

Trace:

1. POST /api/judge/submit

Body: { projectType: "todo", taskId: 1, code: "..." }

2. JudgeController.submitCode()

→ Receives SubmitCodeRequest

3. SubmissionService.submitCode()

→ Creates Submission(id=null, status=PENDING, ...)

→ Saves to DB → id=123 assigned

→ Updates status to RUNNING

4. CodeEvaluationService.evaluateCode("...", "todo", 1)

5. ValidationRuleProvider.getRulesForTask("todo", 1)

→ Returns 7 rules:

1. Check @Entity

2. Check @Id

3. Check @GeneratedValue

4. Check Long id field

5. Check String title field

6. Check String description field

7. Check boolean completed field

6. For each rule, validateRule() is called:

Rule 1: ANNOTATION check for @Entity

→ checkAnnotation(code, "@Entity")

→ code.contains("@Entity") → TRUE

→ TestResult(passed=true, message="✓ Has @Entity annotation")

Rule 2: ANNOTATION check for @Id

→ checkAnnotation(code, "@Id")

→ code.contains("@Id") → TRUE

→ TestResult(passed=true, message="✓ Has @Id annotation")

Rule 4: FIELD check for "Long\s+id"

→ checkField(code, "Long\s+id")

→ Pattern: "(private|public|protected)\s+Long\s+id"

→ pattern.matcher(code).find() → TRUE (finds "private Long id")

→ TestResult(passed=true, message="✓ Has id field of type Long")

... (continues for all rules)

7. checkCompilation(code)

→ Extracts class name: "TodoEntity"

→ Creates /tmp/springboot-judge/TodoEntity.java

- Runs: `javac /tmp/springboot-judge/ToDoEntity.java`
- Exit code: 0 (success)
- `TestResult(passed=true, message="✓ Code compiles successfully")`

8. Calculate results:

- `passedCount = 8` (all rules + compilation)
- `totalTests = 8`
- `allPassed = true`

9. Build output:

" All tests passed! Excellent work!"

- ✓ Has `@Entity` annotation
- ✓ Has `@Id` annotation
- ✓ Has `@GeneratedValue` annotation
- ✓ Has id field of type Long
- ✓ Has title field
- ✓ Has description field
- ✓ Has completed field
- ✓ Code compiles successfully"

10. Update Submission:

- `status = "PASSED"`
- `testsPassed = 8`
- `testsTotal = 8`
- `output = (above message)`
- `evaluatedAt = 2024-01-20T10:30:00`
- Save to DB

11. Create SubmissionResponse:

```
{  
  submissionId: 123,  
  status: "PASSED",  
  output: " All tests passed!...",  
  testsPassed: 8,  
  testsTotal: 8,  
  allPassed: true  
}
```

12. Return to controller

13. Controller returns `ResponseEntity.ok(response)`

14. Client receives 200 OK with response body

Summary

This Spring Boot Online Judge system provides an automated code evaluation platform for students learning Spring Boot. It validates code against predefined rules, checks for proper Java syntax and Spring annotations, and provides immediate feedback. The system is built with clean architecture principles, separating concerns into controllers, services, repositories, and entities, making it maintainable and extensible for additional tasks and project types.